

SW 암호모듈 상세설계서

교과 프로젝트

2026년 6월

Contents

1 문서 개요	2
2 모듈 구성과 파일 구조	2
3 공개 API 설계	2
3.1 생명주기 및 상태 API	3
3.2 암호 서비스 API	3
3.3 주요 자료 크기	3
4 오류 모델과 서비스 인가	3
5 FSM 상세 설계	4
5.1 상태 정의	4
5.2 합법 전이	4
6 초기화와 시작 자가시험	5
6.1 전체 흐름	5
6.2 KAT 구성	5
7 무결성 시험 설계	6
7.1 생성 단계	6
7.2 검증 단계	6
8 엔트로피와 Hash_DRBG 설계	6
8.1 엔트로피 입력	6
8.2 건전성 시험	6
8.3 DRBG 상태와 연산	6
9 ARIA, SHA3, HMAC 내부 설계	7
9.1 ARIA 모드 처리	7
9.2 SHA3 문맥	7

9.3 HMAC-SHA3	7
10 P-256 ECDH 상세 설계	7
11 ML-KEM-768 통합 설계	7
12 영점화와 종료	8
13 공유 라이브러리와 심볼 통제	8
14 시험 설계와 시험 사례	8
15 빌드, 실행 및 감사 절차	9
16 잔여 위험과 설계 한계	9

1 문서 개요

본 문서는 기존 ARIA/SHA3/HMAC 교과 코드를 중심으로 확장한 소프트웨어 암호모듈의 내부 구조와 인터페이스를 파일 및 제어 흐름 단위로 설명한다. 대상 산출물은 Linux 공유 라이브러리 `build/libkcmvp_crypto.so`이다.

본 구현은 교육 목적의 교과 프로젝트이며 공식 KCMVP 또는 FIPS 인증을 받은 모듈이 아니다. 상세 설계의 시험 절차와 상태 통제는 현재 코드의 동작을 기술한 것이며 인증 요구사항 충족을 공식 주장하지 않는다.

2 모듈 구성과 파일 구조

파일 또는 디렉터리	책임
<code>include/kcmvp.h</code>	공개 오류 코드, 상태, 크기 상수와 승인 API 선언
<code>lib/KCMVP_Crypto.c</code>	공유 라이브러리 경계, 입력 검사, 서비스 인가, 초기화와 영점화
<code>lib/module_state.c</code>	비공개 FSM 상태와 합법 전이표
<code>lib/selftest.c</code>	시작 시 ARIA/SHA3/HMAC/DRBG/ECC/ML-KEM KAT
<code>lib/integrity.c</code>	생성 매니페스트의 SHA3-256 무결성 검증
<code>lib/integrity_manifest.h</code>	보호 대상 경로와 기대 SHA3-256 값
<code>lib/entropy.c</code>	Linux <code>getrandom()</code> 및 RCT/APT
<code>lib/hash_drbg.c</code>	SHA3-256 Hash_DRBG 코어
<code>lib/aria.c, lib/aria_Mode.c</code>	기존 ARIA 기본 연산과 ECB/CBC/CTR
<code>lib/sha3.c</code>	호출자 소유 문맥 기반 SHA3/SHAKE 코어
<code>lib/KISA_HMAC_SHA3.c</code>	SHA3 문맥을 사용하는 HMAC
<code>lib/ecc.c</code>	P-256 검증, 공개키, ECDH 및 조건부 시험 어댑터
<code>lib/mlkem.c</code>	ML-KEM API 어댑터, KAT 및 조건부 시험
<code>lib/mlkem_randombytes.c</code>	PQClean <code>randombytes</code> 를 모듈 DRBG에 연결
<code>third_party/micro-ecc</code>	P-256 참조 구현과 BSD 2-Clause 라이선스
<code>third_party/PQClean</code>	ML-KEM-768 clean 구현, Keccak 의존 파일, CC0 라이선스
<code>lib/kcmvp.map</code>	동적 공개 심볼을 <code>KCMVP_*</code> 로 제한
<code>app/main.c</code>	공개 API만 사용하는 번호식 최종 시연 프로그램
<code>test/module_state_test.c</code>	상태, 오류, 장애 주입 및 서비스 통합 시험
<code>test/sha3_vs.c</code>	SHA3/HMAC 벡터 시험
<code>app/kat.c</code>	ARIA 벡터 파서와 암호화/복호화 검증

3 공개 API 설계

외부 사용자는 `include/kcmvp.h`를 포함한다. 모든 공개 서비스는 정확한 포인터와 길이를 검사하고 모듈 상태 및 해당 시작 KAT 플래그를 확인한다.

3.1 생명주기 및 상태 API

API	기능
KCMVP_Initialize	시작 시험과 운영 DRBG 초기화 후 NORMAL 진입
KCMVP_GetState	현재 상태 열거값 반환
KCMVP_GetStatus	상태, 초기화 여부, 운영 가능 여부 반환
KCMVP_Zeroize	내부 시험 플래그와 DRBG 제거 후 오류 상태로 전이
KCMVP_Shutdown	내부 상태를 제거하고 EXIT로 전이
KCMVP_PreSelfTest	호환용 사전 시험 진입점; PRE_SELFTEST에서만 허용
KCMVP_AlgKATSelfTest	호환용 알고리즘 KAT 진입점; PRE_SELFTEST에서만 허용

3.2 암호 서비스 API

API 그룹	기능
KCMVP_ARIA_*	키 스케줄 호환 함수, ECB/CBC/CTR 암호화, 기본 블록 연산
KCMVP_SHA3_Hash	SHA3-224/256/384/512 단일 호출 해시
KCMVP_HMAC_SHA3	HMAC-SHA3-224/256/384/512
KCMVP_RNG_Generate	운영 Hash_DRBG 출력 생성
KCMVP_RNG_Reseed	새 엔트로피로 운영 DRBG 재시드
KCMVP_RNG_Uninstantiate	운영 DRBG 상태 제거
KCMVP_ECC_GenerateKeyPair	DRBG 기반 P-256 키 쌍 생성 및 조건부 시험
KCMVP_ECC_ComputeSharedSecret	상대 공개키 검증 후 P-256 ECDH
KCMVP_MLKEM_Keypair	ML-KEM-768 키 쌍 생성 및 조건부 시험
KCMVP_MLKEM_Encaps	공개키로 암호문과 공유 비밀 생성
KCMVP_MLKEM_Decaps	비밀키로 공유 비밀 복원; PQClean 암시적 거부 사용

3.3 주요 자료 크기

알고리즘	객체	바이트 수
P-256	개인키 / 공개키 / 공유 비밀	32 / 64 / 32
ML-KEM-768	비밀키 / 공개키	2400 / 1184
ML-KEM-768	암호문 / 공유 비밀	1088 / 32

4 오류 모델과 서비스 인가

오류 코드	의미
KCMVP_SUCCESS	성공
KCMVP_ERROR_GENERIC	일반 실패
KCMVP_ERROR_NOT_INITIALIZED	LOAD 상태에서 서비스 호출
KCMVP_ERROR_INVALID_STATE	현재 상태에서 허용되지 않는 호출
KCMVP_ERROR_SELF_TEST	시작 자가시험 또는 엔트로피 시험 실패
KCMVP_ERROR_INVALID_PARAM	포인터, 길이, 모드, 키 크기 등 오류
KCMVP_ERROR_CRYPT0	내부 암호 연산 실패
KCMVP_ERROR_RESEED_REQUIRED	DRBG 재시드 간격 초과
KCMVP_ERROR_INVALID_PUBLIC_KEY	P-256 상대 공개키 유효성 실패
KCMVP_ERROR_CONDITIONAL_TEST	키 생성 후 조건부 자가시험 실패

일반 서비스는 `authorize_algorithm()`을 통해 다음 절차를 따른다.

1. `module_authorize_service()`가 현재 상태를 확인한다.
2. NORMAL이면 EXECUTION으로 전이한다.
3. 해당 알고리즘 KAT 통과 플래그를 검사한다.
4. 입력을 검증하고 내부 연산을 수행한다.
5. `module_complete_service()`로 NORMAL에 복귀한다.

P-256과 ML-KEM 키 생성은 일반 실행 상태 대신 키 생성 뒤 COND_SELFTEST 전이를 포함하므로 별도의 제어 흐름을 사용한다.

5 FSM 상세 설계

5.1 상태 정의

상태	의미
LOAD	프로세스 시작 후 미초기화 상태
PRE_SELFTEST	무결성, 엔트로피, KAT 수행 상태
NORMAL	승인된 공개 서비스를 받을 수 있는 운영 상태
EXECUTION	일반 공개 서비스 수행 중 상태
COND_SELFTEST	생성 키의 조건부 시험 수행 상태
CRITICAL_ERROR	암호 서비스가 차단된 비복구 오류 상태
EXIT	종료 완료 상태
DEGRADED, NORMAL_ERROR	예약 상태; 현재 경로에서 사용하지 않음

5.2 합법 전이

현재 상태	이벤트	다음 상태
LOAD	BEGIN_SELFTEST	PRE_SELFTEST
LOAD	FATAL_ERROR	CRITICAL_ERROR
LOAD	SHUTDOWN	EXIT

현재 상태	이벤트	다음 상태
PRE_SELFTEST	SELFTEST_PASSED	NORMAL
PRE_SELFTEST	FATAL_ERROR	CRITICAL_ERROR
NORMAL	BEGIN_SERVICE	EXECUTION
NORMAL	BEGIN_COND_SELFTEST	COND_SELFTEST
NORMAL	FATAL_ERROR	CRITICAL_ERROR
NORMAL	SHUTDOWN	EXIT
EXECUTION	SERVICE_COMPLETE	NORMAL
EXECUTION	FATAL_ERROR	CRITICAL_ERROR
COND_SELFTEST	COND_SELFTEST_PASSED	NORMAL
COND_SELFTEST	FATAL_ERROR	CRITICAL_ERROR
CRITICAL_ERROR	SHUTDOWN	EXIT

표에 없는 전이는 KCMVP_ERROR_INVALID_STATE로 거부한다. 상태 변수는 파일 범위 static이며 공개 쓰기 전역 변수가 아니다.

6 초기화와 시작 자가시험

6.1 전체 흐름

```
KCMVP_Initialize
  reset test flags and DRBG
  LOAD -> PRE_SELFTEST
  integrity_verify
  entropy_startup_health_test
  ARIA KAT
  SHA3 KAT
  HMAC-SHA3 KAT
  Hash_DRBG KAT
  P-256 KAT
  ML-KEM-768 KAT
  instantiate operational Hash_DRBG
  PRE_SELFTEST -> NORMAL
```

실패 시 운영 DRBG와 자가시험 플래그를 제거하고 FATAL_ERROR 이벤트로 CRITICAL_ERROR에 진입한다. 알고리즘별 플래그는 해당 KAT가 성공한 후에만 1로 설정한다.

6.2 KAT 구성

- ARIA: 고정 키, 평문, 암호문으로 ARIA-128 암호화와 복호화 확인
- SHA3: 문자열 abc의 SHA3-256 기대값 비교
- HMAC-SHA3: 고정 키/메시지의 HMAC-SHA3-224 기대값 비교
- Hash_DRBG: 고정 48바이트 시드로 인스턴스화 후 64바이트 기대값 비교
- P-256: 고정 개인키 2, 3의 공개키와 ECDH 공유 비밀 비교
- ML-KEM-768: 고정 key coins와 encapsulation coins를 사용하는 결정론적 키 생성/캡슐화/복호화 및 공유 비밀 기대값 비교

7 무결성 시험 설계

7.1 생성 단계

tools/generate_integrity_manifest.c는 빌드 시 보호 대상으로 선택한 모듈 소스 파일을 읽고 각 파일의 SHA3-256을 계산한다. 결과는 경로와 32바이트 다이제스트의 배열인 lib/integrity_manifest.h로 생성된다. Makefile의 MANIFEST_INPUTS 변경 시 매니페스트가 다시 생성된다.

7.2 검증 단계

초기화의 PRE_SELFTEST에서 integrity_verify()가 매니페스트의 각 파일을 다시 해시한다. 실제 값과 기대값은 누적 XOR 방식으로 비교한다. 파일 열기, 읽기, 해시 또는 비교 실패는 시작 자가시험 실패로 처리한다.

이 설계는 프로젝트 파일의 우발적 변경과 단순 변조를 탐지하기 위한 것이다. 서명된 매니페스트, 신뢰 루트, 로더 검증 또는 메모리 실행 페이지 검증을 제공하지 않으며 프로젝트 루트 기준 경로에 의존한다.

8 엔트로피와 Hash_DRBG 설계

8.1 엔트로피 입력

Linux에서는 getrandom()만 사용하며 rand() 등으로 조용히 대체하지 않는다. 시작 시험은 512바이트를 수집한다. 시험 빌드에서만 mock provider를 주입할 수 있으며 프로덕션 공개 API로 노출되지 않는다.

8.2 건전성 시험

- RCT: 같은 바이트가 연속되는 횟수를 검사하며 cutoff는 5이다.
- APT: 64바이트 창에서 기준 심볼의 출현 빈도를 검사하며 cutoff는 16이다.
- 공급자 실패, RCT 실패 또는 APT 실패는 초기화를 실패시킨다.

이 임계값은 교과 프로젝트 설정이며 실측 최소 엔트로피 분석을 통한 공식 파라미터 선정 결과가 아니다.

8.3 DRBG 상태와 연산

HASH_DRBG_CTX는 55바이트 V, 55바이트 C, 64비트 재시드 카운터와 인스턴스 플래그를 가진다. 주요 설계값은 다음과 같다.

항목	값
해시	SHA3-256
시드 길이	440비트 (55바이트)
인스턴스 엔트로피	32바이트
nonce 길이	16바이트
최대 생성 요청	65,536바이트
재시드 간격	2^{48} 생성 요청

인스턴스화와 재시드는 `entropy_get()`을 사용한다. 생성 후에는 `Hash_DRBG` 규칙에 따라 `v`와 카운터를 갱신한다. 해제, 모듈 영점화, 종료 및 치명적 재시드 실패 시 문맥 전체를 지운다. ECC와 ML-KEM 내부 난수도 이 운영 DRBG를 사용한다.

9 ARIA, SHA3, HMAC 내부 설계

9.1 ARIA 모드 처리

`KCMVP_ARIA_Crypt`는 방향, 모드, 키, IV, 입출력 포인터, 키 비트 수와 길이를 검사한다. ECB/CBC는 16바이트 배수만 허용한다. CTR은 암호화와 복호화 모두 ARIA 암호화 키 스케줄을 사용하고 마지막 부분 블록을 처리한다. CBC 복호화는 현재 암호문 블록을 보존하므로 동일 입출력 버퍼 사용에도 체인 값이 손상되지 않는다.

9.2 SHA3 문맥

`SHA3_CTX`는 200바이트 Keccak 상태, rate, capacity, suffix, 현재 offset과 초기화 정보를 포함한다. 전역 가변 해시 상태를 사용하지 않는다. `sha3_init/update/final`은 문맥 포인터와 입출력 길이를 검증하고 `final` 후 문맥을 영점화한다. 공개 API는 단일 호출 해시만 제공하고 원시 SHA3 함수는 공유 라이브러리 외부에 노출하지 않는다.

9.3 HMAC-SHA3

긴 키는 선택한 SHA3 변형으로 먼저 축약한다. 내부 해시와 외부 해시는 각각 독립 `SHA3_CTX`를 사용한다. 공개 경계는 메시지/키 포인터 조건과 224/256/384/512 비트 선택을 검사한다.

10 P-256 ECDH 상세 설계

`micro-ecc`는 `secp256r1`만 빌드하도록 컴파일 플래그를 제한한다. `micro-ecc`의 자체 RNG 경로는 사용하지 않으며 모듈 공개 키 생성 흐름에서 `Hash_DRBG`로 32바이트 개인키 후보를 생성한다. 유효한 공개키가 생성될 때까지 최대 64회 시도한다.

키 쌍 생성 후 다음 조건부 시험을 수행한다.

1. `NORMAL`에서 `COND_SELFTEST`로 전이한다.
2. 고정 유효 P-256 peer 공개키와 생성 개인키로 공유 비밀을 구한다.
3. 고정 peer 개인키와 생성 공개키로 역방향 공유 비밀을 구한다.
4. 두 32바이트 결과를 비교한다.
5. 성공 시 `NORMAL`로 복귀하고, 실패 시 키를 지운 뒤 `CRITICAL_ERROR`로 전이한다.

ECDH 서비스는 상대 공개키 좌표가 곡선 위의 유효한 점인지 확인한다. 반환값은 32바이트 x 좌표이며 응용 키로 직접 사용하기보다 적절한 KDF 적용이 필요하다.

11 ML-KEM-768 통합 설계

PQClean 저장소 커밋 202a8f96315f9ed219387a50f7e40d04af037ea8의 `crypto_kem/ml-kem-768/clean` 구현만 빌드한다. 필요한 공통 Keccak 구현과 헤더만 함께 vendoring하였다. 원본 구현 라이선스는 CC0/public domain이다.

PQClean의 PQCLEAN_randombytes는 lib/mlkem_randombytes.c를 통해 운영 Hash_DRBG의 내부 생성 경로로 연결된다. 공개 KCMVP_RNG_Generate를 중첩 호출하지 않으므로 서비스 FSM이 중복 진입하지 않는다. RNG 실패는 PQClean keypair/encaps 함수의 실패로 전파된다.

ML-KEM 키 생성 조건부 시험은 고정 32바이트 coins를 사용하여 생성 공개키로 결정론적 캡슐화를 수행하고, 생성 비밀키로 복호화한 32바이트 공유 비밀과 비교한다. 실패 시 공개키와 비밀키를 지우고 CRITICAL_ERROR로 전이한다. 일반 decapsulation은 PQClean 구현의 implicit rejection 동작을 유지한다.

12 영점화와 종료

secure_zero()는 volatile 바이트 포인터로 메모리를 덮어써 컴파일러가 제거하기 어렵게 한다. 다음 객체가 내부적으로 영점화된다.

- 운영 Hash_DRBG 문맥과 재시드 카운터
- 알고리즘 KAT 통과 플래그
- ARIA 라운드 키와 모드 임시 블록
- SHA3 최종화 문맥과 DRBG 해시 임시값
- ECC 조건부 시험 공유 비밀 및 실패한 후보 키
- ML-KEM KAT/조건부 시험 임시 키, 암호문, 공유 비밀

KCMVP_Zeroize()는 내부 모듈 데이터를 제거하고 정상 운영 상태라면 CRITICAL_ERROR로 전이하여 이후 암호 서비스를 차단한다. KCMVP_Shutdown()은 데이터를 제거하고 최종적으로 EXIT에 진입한다. 이미 호출자에게 반환한 개인키와 공유 비밀은 호출자가 별도로 영점화해야 한다.

13 공유 라이브러리와 심볼 통제

라이브러리는 -fvisibility=hidden으로 컴파일하고 -Wl,--version-script=lib/kcmvp.map으로 연결한다. 버전 스크립트는 KCMVP_*만 global로 지정하고 나머지를 local로 처리한다. 따라서 ARIA, SHA3, HMAC, DRBG, 엔트로피, 무결성, micro-ecc 및 PQClean 원시 심볼은 동적 공개 API가 아니다.

make audit-exports는 nm -D --defined-only 결과에서 이름이 KCMVP_로 시작하지 않는 동적 정의 심볼을 탐지하면 실패한다. app/main.c는 공유 라이브러리에 연결되며 승인된 공개 API만 호출한다.

14 시험 설계와 시험 사례

분류	주요 시험 사례
상태/FSM	초기화 전 거부, NORMAL 진입, 오류 상태 차단, shutdown 후 EXIT
무결성	정상 매니페스트 성공, 기대 태그 변조 시 초기화 실패
엔트로피	정상 공급자, 반복 출력 RCT 실패, 편향 출력 APT 실패, 공급자 실패
시작 KAT	각 알고리즘 성공 및 강제 실패 시 CRITICAL_ERROR
ARIA	ECB/CBC/CTR 암호화, 부분 CTR, CBC 비정렬 거부, NULL/키/방향 검사
SHA3/HMAC	1,266개 벡터, 독립 문맥, 반복 호출 및 잘못된 파라미터

분류	주요 시험 사례
DRBG	KAT, 생성, 재시드, 최대 요청, 해제, 엔트로피 실패
P-256	키 쌍, 양방향ECDH 일치, 잘못된 공개키, 조건부 시험 실패
ML-KEM	KAT, 키 쌍/캡슐화/복호화 일치, 길이 검사, 조건부 시험 실패
ARIA 벡터	ECB/CBC/CTR의 3,342개 벡터에서 암호화와 복호화 확인
심볼	승인된 KCMVP_* 동적 심볼만 존재하는지 감사

장애 주입 함수는 시험 전용 컴파일 매크로에서만 활성화되며 프로덕션 app/main.c에는 노출하지 않는다. 최종 시연 프로그램은 장애 주입 항목에 대해 build/module_state_test 실행을 안내한다.

15 빌드, 실행 및 감사 절차

모든 명령은 프로젝트 루트에서 실행한다.

```
# 공유라이브러리, 데모, 시험실행파일빌드
make clean
make

# 공개API 최종시연
./build/main
./build/main --all

# 개별시험
./build/module_state_test
./build/main --vectors
./build/sha3_vs

# 전체시험과공개심볼확인
make test
make audit-exports

# clean rebuild + all tests + export audit
make clean && make audit

# 변경파일의공백패치/ 형식확인
git diff --check
```

빌드 시 보호 대상 소스가 변경되면 무결성 매니페스트가 먼저 재생성되고 그 다음 공유 라이브러리가 링크된다. 실행 파일의 rpath는 \$ORIGIN으로 설정되어 같은 build/ 디렉터리의 공유 라이브러리를 찾는다.

16 잔여 위험과 설계 한계

- 공식 암호모듈 검증, 운영 환경 형상 승인, 보안 경계 인증을 수행하지 않았다.
- 소스 기반 무결성 매니페스트는 서명되지 않았고 실행 코드 페이지를 검증하지 않는다.
- 엔트로피 건전성 임계값은 프로젝트 기준이며 엔트로피 원천의 통계적 최소 엔트로피 평가를 대체하지 않는다.
- 원시 ECDH 공유 비밀에 대한 KDF는 응용 계층의 책임이다.
- 프로세스 수준 동시 호출 정책과 전체 모듈 잠금은 별도 설계하지 않았다. SHA3는 문맥화되었지만 모듈 FSM과 단일 운영 DRBG는 모듈 인스턴스 상태이다.
- 외부 참조 구현은 고정 커밋과 라이선스를 기록했으나 장기 유지보수와 취약점 모니터링은 별도로 수행해야 한다.
- 호출자 버퍼에 반환된 비밀 자료의 보관, 접근 통제, 사용 후 삭제는 호출자 책임이다.